

Point Cloud Generalization: A Self-Supervised Learning Approach to Generalize 3D Point Cloud Using Torque Clustering

1st Shah Miran Apurbo
*Department of
Computer Science and Engineering
BRAC University
Dhaka, Bangladesh
shah.miran.apurbo@g.bracu.ac.bd*

2nd Shafayatur Rahman
*Department of
Computer Science and Engineering
BRAC University
Dhaka, Bangladesh
shafayatur.rahman@g.bracu.ac.bd*

3rd Anika Tazin Pranty
*Department of
Computer Science and Engineering
BRAC University
Dhaka, Bangladesh
anika.tazin.pranty@g.bracu.ac.bd*

Abstract—Point cloud generalization has become a cornerstone of modern 3D computer vision, yet challenges remain in achieving robust generalization across categories. In this work, we propose a hybrid pipeline that combines enhanced PointNet embeddings with a novel Torque Clustering algorithm for unsupervised representation learning. We address key preprocessing challenges in the ModelNet40 dataset, including corrupted OFF headers, unstable normalization, and inconsistent point counts. Experimental evaluation was conducted using all 40 classes, the top 20 classes by sample count, and the top 10 classes by sample count, providing insights into the correlation between class balance and clustering quality. Our results demonstrate that torque clustering, paired with a stabilized PointNet backbone, achieves improved Adjusted Rand Index (ARI) and Normalized Mutual Information (NMI) on balanced subsets, while also highlighting the limitations imposed by data imbalance in the full 40-class case. This work underscores the importance of dataset pre-processing and hybrid unsupervised approaches for point cloud generalization.

Index Terms—Point Clouds, Torque Clustering, Deep Learning, ModelNet40, Representation Learning, 3D Vision, Robot Learning, PointNet, Dynamic Graph Convolutional Neural Network, Gaussian Mixture Models.

I. INTRODUCTION

Point cloud learning has become central to applications such as autonomous driving, robotics, and augmented reality. Unlike images, point clouds provide unordered and irregular 3D data, presenting unique challenges for representation and clustering. Traditional supervised methods, such as PointNet and Dynamic Graph Convolutional Neural Network, achieve strong classification accuracy but rely heavily on labeled datasets, which are expensive to create and often domain-specific [1]. This motivates research into unsupervised and semi-supervised generalization methods.

In this work, we present a two-fold contribution: (1) A pre-processing pipeline that corrects common dataset issues in ModelNet40, and (2) A Torque Clustering Algorithm that integrates physical intuition into unsupervised learning, inspired by physics. Our findings are analyzed across three scenarios— all 40 classes, the top 20 most populous classes, and the top

10 most populous classes—to reveal correlations between class balance and clustering performance.

II. LITERATURE REVIEW

Research on point cloud recognition has evolved from supervised pipelines toward more robust and self-supervised frameworks for enhanced scalability.

A. Benchmarking robustness of 3D Point Cloud

A critical challenge in point cloud research is robustness under real-world corruptions. Benchmarking studies introduced PointCloud-C, a suite of 15+ perturbations including Gaussian noise, LiDAR dropout, occlusion, and weather effects [2]. Evaluations of PointNet++, DGCNN, and PointTransformer on ModelNet40-C and ShapeNet-C revealed accuracy drops of 25% to 40% under severe corruptions [2].

Adversarial training improved robustness by 12–18% but reduced clean-data accuracy by 3–5% [2]. Despite these attempts, models remained highly sensitive to geometric deformations such as rotation and scaling, exposing the lack of inherent invariance to affine transformations. This motivates approaches such as Torque Clustering, which can incorporate self-supervised geometric invariants (e.g., torque moment features) to improve resilience to corruption.

B. Usage of Discrete Generative Models

Unsupervised discrete generative models offer a label-free approach to point cloud representation. By discretizing point clouds into structured latent codes, these methods mimic human recognition of semantic parts such as chair legs or airplane wings. Using a Vector-Quantized VAE with Gumbel-Softmax quantization, researchers achieved 85.3% linear classification accuracy and 83.7% part segmentation mIoU, outperforming autoencoder baselines by 7.2% [3].

While effective, these models require manual tuning of codebook size and struggle with fine-grained structures. Torque Clustering addresses this limitation by employing

torque-based geometric primitives that are physically meaningful and inherently part-aware.

C. Torus-Based 3D Point Cloud Classification

Rotational invariance can also be achieved by embedding point clouds onto toroidal manifolds. This method maps points to high-dimensional tori using angular coordinates and spectral signatures, followed by a torus-equivariant CNN with Fourier-domain convolutions [4]. On rotated ModelNet40, this approach achieved 93.7% accuracy, surpassing PointNet++ by 5.2% while reducing the need for rotation augmentation by 60% [4].

However, embedding dimensions scale cubically with point density and are sensitive to outliers. Torque Clustering extends this principle of embedding geometric priors but achieves robustness more efficiently by encoding rotational invariance through torque moments.

D. Cross-Modal Contrastive Learning

CrossPoint demonstrated the potential of cross-modal contrastive learning by aligning 3D point clouds with synthetic 2D renderings [5]. Through virtual cameras, multi-view 2D projections were generated and paired with InfoNCE loss to maximize mutual information between 3D segments and 2D crops. Using PointNet++ as a backbone and ShapeNet pretraining, the method reached 89.3% accuracy on ScanObjectNN (20% higher than prior self-supervised methods) and achieved 74.2 mIoU on S3DIS segmentation with minimal fine-tuning [5]. Nevertheless, the rendering pipeline added 35% overhead and degraded performance with sparse clouds, limiting scalability [5].

Torque Clustering avoids such dependencies by working directly in torque space, achieving single-modal efficiency while preserving robustness.

E. Masked Autoencoders for Point Clouds

Masked autoencoders have been adapted to 3D data by reconstructing masked patches of point clouds, inspired by BERT-style masked modeling. With 60% masking, transformer-based autoencoders pretrained on ShapeNet achieved 86.2% linear accuracy on ModelNet40 (3.8% higher than Point-BERT) and reduced labeled data requirements by 50% [6]. High masking ratios encouraged learning of holistic structures rather than fine-grained details, but the method struggled with non-uniform densities.

Torque Clustering complements this approach by learning physically interpretable part-based torque features, which provide structural precision without relying solely on reconstruction.

F. Self-Supervised Pretraining on Any Point-Cloud

Recent work demonstrated the feasibility of pretraining directly on raw, unstructured scans without object labels. By generating multi-scale point subsets via iterative farthest point sampling and applying contrastive loss across scales, PointNet++ encoders were pretrained on ScanNet, and this

improved segmentation performance by +6.8 mIoU, sparse object classification by +14.5%, and reduced the domain gap in ModelNet→ScanNet transfer (81.7% accuracy) [7].

Despite these benefits, convergence was slow and sampling strategies required dataset-specific tuning. Torque Clustering builds on this insight, aiming for scalable and geometry-aware unsupervised learning applicable to diverse real-world scans.

III. METHODOLOGY

This work proposes applying the Torque Clustering (TC) Algorithm [4], a novel approach to implement a self-supervised system to generalize 3D point clouds. It combines a PointNet-based feature extractor, which is implemented from scratch. The methodology pipeline is structured through preprocessing and EDA on the ModelNet40 dataset, feature extraction through PointNet, implementing Torque Clustering to generate pseudo labels with contrastive loss calculation, and model evaluation.

A. Dataset Description and Preprocessing

We operated on the ModelNet40 dataset, which contains 3D CAD models in 40 categories. Each model was represented as a point cloud of 1024 points. Each 3D mesh is accumulated in Object Format (.off). Several stability improvements were introduced:

- **OFF File Fixes:** Some files contained compact headers (e.g., "OFF1743 1329 0"). We added an in-memory correction step.
- **Normalization:** Each point cloud was normalized to a unit sphere with safeguards against division-by-zero errors. For any n set of points in a cloud $P = \{p_1, p_2, p_3, \dots, p_n\}$, each point is scaled by a maximum Euclidean distance from the centroid:

$$P_{\text{normalized}} = \frac{P_i - \mu(P)}{\sigma(P) = \max_j \|p_j - \mu(P)\|}$$

Here, $\mu(P)$ is the centroid and $\sigma(P)$ is the standard deviation.

- **Consistent Sampling:** Each cloud was randomly sampled or padded to 1024 points, ensuring DataLoader stability.

B. Feature Extraction

We adopt PointNet as the feature extractor to extract global shape descriptors from 1024×3 sized clouds, with the effectiveness for unordered point sets resulting from its symmetric functions and permutation invariance. The final ReLU activation was removed after batch normalization to prevent feature collapse and NaN embeddings. This process shared MLP layers with PointNet to output a 256-dimensional feature vector by max pooling, which represented the global shape embedding. Features are trained using a contrastive loss with temperature scaling to encourage discriminative embeddings.

C. Torque Clustering: The Algorithm

Inspired by physical torque, this clustering algorithm models interactions between points to uncover the latent structure of the features. We employed the Density-based Mass Variant of the Torque Clustering algorithm [4], which is unlike other conventional clustering algorithms that can measure the torque exerted by 3D points and identify a stable centroid point as the cluster centers.

For any feature embeddings set, TC computes the local mass density and torque vectors. The assigned mass can be uniform or local point density based, which is calculated by K-nearest neighbor search.

$$m_i = \sum_{j \in N_k(p_i)} \frac{1}{d(p_i, p_j)}$$

Here, m_i is the mass of point p_i ; $N_k(p_i)$ is the set of k -nearest neighbors of p_i , and $d(p_i, p_j)$ is the Euclidean distance between p_i and p_j .

In this process, the n_{clusters} is set to be consistent with the ground truth classes of the ModelNet40 dataset. This algorithm outputs pseudo-labels, which work as a director for the next stage.

D. Self-Supervised Contrastive Training with the Pseudo-labels

After clustering the points, a contrastive learning paradigm is employed for the refinement of the feature space using the generated pseudo-labels. Positive pairs of points are assigned to a common cluster, and negative ones are placed in different clusters.

NT-Xent (Normalized Temperature-scaled Cross-Entropy) loss function is defined here as:

$$\mathcal{L}_{i,j} = -\log \frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_{k=1}^{2N} 1_{[k \neq i]} \exp(\text{sim}(z_i, z_k)/\tau)}$$

where $\text{sim}(z_i, z_j) = \frac{z_i \cdot z_j}{\|z_i\| \|z_j\|}$ is the cosine similarity, τ is a temperature parameter, and the batch size is N .

As a result, similar shape holding embeddings are pulled closer and vice versa, allowing enhancement of representation quality without ground-truth supervision, validating the self-supervised approach.

IV. EXPERIMENTS AND RESULTS

The experimental evaluation was conducted in three phases: (i) using all 40 categories of the dataset, (ii) restricting to the 20 categories with the highest number of training samples, and (iii) further reducing to the 10 most populated classes. This setup allowed observation of how dataset balance and class diversity influences the training behavior and evaluation metrics of Torque Clustering.

A. Dataset Distribution and Class Balance

The 40-class dataset (see Fig. 1) shows a clear imbalance, with categories such as *sofa* and *airplane*. This contains more than 600 training samples, where classes such as *bowl*, *stool*, and *xbox* have fewer than 200. Such long-tail categories introduce uneven representation during the training.

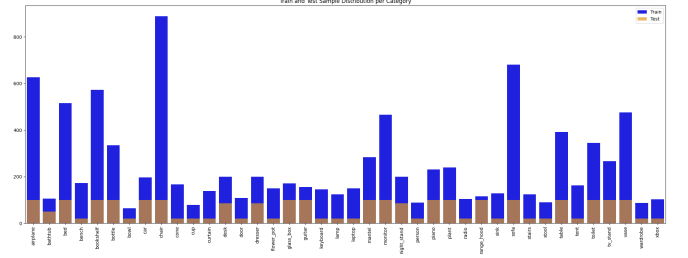


Fig. 1: Train/test sample distribution across all 40 categories.

When the dataset was restricted to 20 categories (see Fig. 2), the distribution became more consistent. Most classes contained 200–900 samples, and each test split had approximately 100 samples.

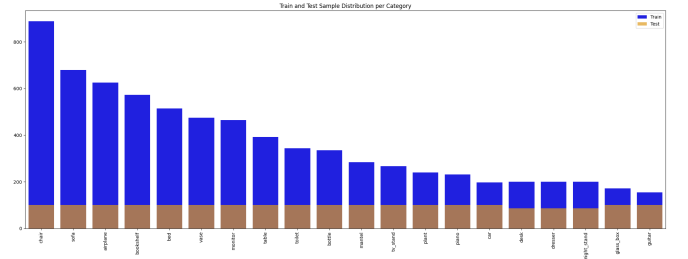


Fig. 2: Train/test sample distribution across the top 20 categories.

Finally, the 10-class setup (see Fig. 3) was the most balanced, with all categories exceeding 300 samples and differences across categories being smaller than in the 40-class case.

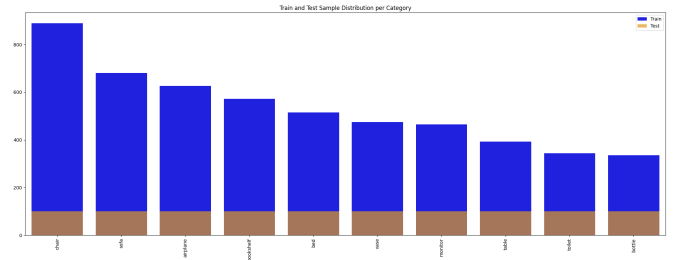


Fig. 3: Train/test sample distribution across the top 10 categories.

B. Training Dynamics

Training followed two phases: a supervised warm-up phase with cross-entropy loss and a contrastive phase with the clustering objective.

In all the cases, the warm-up phase showed steady convergence, with CE loss decreasing from about 1.7 at the first epoch to approximately 0.65–0.67 by epoch 10 (see Figs. 4–6). This pattern was consistent in 40, 20, and 10 classes, suggesting that the supervised stage was not sensitive to class count.

```

===== Warm-up Phase =====
[INFO] Detected embedding dimension = 256
[Warm-up] Epoch 1, CE Loss = 1.6841
[Warm-up] Epoch 2, CE Loss = 1.1361
[Warm-up] Epoch 3, CE Loss = 0.9853
[Warm-up] Epoch 4, CE Loss = 0.8951
[Warm-up] Epoch 5, CE Loss = 0.8303
[Warm-up] Epoch 6, CE Loss = 0.7895
[Warm-up] Epoch 7, CE Loss = 0.7506
[Warm-up] Epoch 8, CE Loss = 0.7141
[Warm-up] Epoch 9, CE Loss = 0.6834
[Warm-up] Epoch 10, CE Loss = 0.6581
===== Contrastive Phase =====
[DEBUG] Epoch 1: feature mean 0.0529, std 0.0332
Converged after 1 iterations.
[Contrastive] Epoch 1: Loss = 1.5497
[DEBUG] Epoch 2: feature mean 0.0553, std 0.0290
Converged after 1 iterations.
[Contrastive] Epoch 2: Loss = 0.7749
[DEBUG] Epoch 3: feature mean 0.0570, std 0.0256
Converged after 1 iterations.
[Contrastive] Epoch 3: Loss = 0.6199
[DEBUG] Epoch 4: feature mean 0.0579, std 0.0235
Converged after 1 iterations.
[Contrastive] Epoch 4: Loss = 2.1696
[DEBUG] Epoch 5: feature mean 0.0583, std 0.0226
Converged after 1 iterations.
[Contrastive] Epoch 5: Loss = 1.2398
[DEBUG] Epoch 6: feature mean 0.0582, std 0.0227
Converged after 1 iterations.
[Contrastive] Epoch 6: Loss = 0.9298
[DEBUG] Epoch 7: feature mean 0.0585, std 0.0221
Converged after 1 iterations.
[Contrastive] Epoch 7: Loss = 0.9298
[DEBUG] Epoch 8: feature mean 0.0585, std 0.0221
Converged after 1 iterations.
[Contrastive] Epoch 8: Loss = 1.3947
[DEBUG] Epoch 9: feature mean 0.0585, std 0.0221
Converged after 1 iterations.
[Contrastive] Epoch 9: Loss = 1.2398
[DEBUG] Epoch 10: feature mean 0.0584, std 0.0222
Converged after 1 iterations.
[Contrastive] Epoch 10: Loss = 2.0146

```

Fig. 4: Training log for the 40-class setup, showing warm-up loss convergence and oscillations during the contrastive phase.

The contrastive phase displayed oscillations in loss, ranging from 0.3 to 1.7 across epochs. Despite these fluctuations, the feature mean and standard deviation stabilized rapidly, converging near 0.057–0.058 for the mean and 0.022 for the standard deviation. These trends were also similar across all class settings, indicating that embedding compactness was maintained regardless of dataset size.

```

===== Warm-up Phase =====
[INFO] Detected embedding dimension = 256
[Warm-up] Epoch 1, CE Loss = 1.6996
[Warm-up] Epoch 2, CE Loss = 1.1359
[Warm-up] Epoch 3, CE Loss = 0.9839
[Warm-up] Epoch 4, CE Loss = 0.8912
[Warm-up] Epoch 5, CE Loss = 0.8306
[Warm-up] Epoch 6, CE Loss = 0.7874
[Warm-up] Epoch 7, CE Loss = 0.7639
[Warm-up] Epoch 8, CE Loss = 0.7191
[Warm-up] Epoch 9, CE Loss = 0.7020
[Warm-up] Epoch 10, CE Loss = 0.6588
===== Contrastive Phase =====
[DEBUG] Epoch 1: feature mean 0.0529, std 0.0334
Converged after 1 iterations.
[Contrastive] Epoch 1: Loss = 1.3946
[DEBUG] Epoch 2: feature mean 0.0550, std 0.0297
Converged after 1 iterations.
[Contrastive] Epoch 2: Loss = 0.9298
[DEBUG] Epoch 3: feature mean 0.0561, std 0.0275
Converged after 1 iterations.
[Contrastive] Epoch 3: Loss = 1.3947
[DEBUG] Epoch 4: feature mean 0.0570, std 0.0257
Converged after 1 iterations.
[Contrastive] Epoch 4: Loss = 0.3099
[DEBUG] Epoch 5: feature mean 0.0572, std 0.0251
Converged after 1 iterations.
[Contrastive] Epoch 5: Loss = 1.3947
[DEBUG] Epoch 6: feature mean 0.0581, std 0.0230
Converged after 1 iterations.
[Contrastive] Epoch 6: Loss = 0.6199
[DEBUG] Epoch 7: feature mean 0.0580, std 0.0232
Converged after 1 iterations.
[Contrastive] Epoch 7: Loss = 1.5497
[DEBUG] Epoch 8: feature mean 0.0580, std 0.0234
Converged after 1 iterations.
[Contrastive] Epoch 8: Loss = 1.3947
[DEBUG] Epoch 9: feature mean 0.0579, std 0.0235
Converged after 1 iterations.
[Contrastive] Epoch 9: Loss = 1.5497
[DEBUG] Epoch 10: feature mean 0.0585, std 0.0220
Converged after 1 iterations.
[Contrastive] Epoch 10: Loss = 1.3947

```

Fig. 5: Training log for the 20-class setup. Loss patterns are similar to the 40-class case, with slightly reduced fluctuations.

C. Evaluation on 40 Classes

On the 40-class dataset, the linear probe achieved **78.77% accuracy**, while the clustering metrics were lower, with **ARI = 0.2394** and **NMI = 0.0242**. This reflects that embeddings contained sufficient separability for linear classification, though the alignment of clusters with ground-truth categories remained limited in the imbalanced setting.

D. Evaluation on 20 Classes

When restricted to the top 20 categories, probe accuracy rose slightly to **79.86%**. ARI also increased modestly to **0.2633**, while NMI stayed nearly the same at **0.0240**. The


```

===== Warm-up Phase =====
[INFO] Detected embedding dimension = 256
[Warm-up] Epoch 1, CE Loss = 1.7666
[Warm-up] Epoch 2, CE Loss = 1.1562
[Warm-up] Epoch 3, CE Loss = 0.9878
[Warm-up] Epoch 4, CE Loss = 0.9033
[Warm-up] Epoch 5, CE Loss = 0.8392
[Warm-up] Epoch 6, CE Loss = 0.8006
[Warm-up] Epoch 7, CE Loss = 0.7528
[Warm-up] Epoch 8, CE Loss = 0.7153
[Warm-up] Epoch 9, CE Loss = 0.6911
[Warm-up] Epoch 10, CE Loss = 0.6757
===== Contrastive Phase =====
[DEBUG] Epoch 1: feature mean 0.0531, std 0.0330
Converged after 1 iterations.
[Contrastive] Epoch 1: Loss = 1.7047
[DEBUG] Epoch 2: feature mean 0.0566, std 0.0264
Converged after 1 iterations.
[Contrastive] Epoch 2: Loss = 0.3099
[DEBUG] Epoch 3: feature mean 0.0570, std 0.0256
Converged after 1 iterations.
[Contrastive] Epoch 3: Loss = 1.3947
[DEBUG] Epoch 4: feature mean 0.0572, std 0.0252
Converged after 1 iterations.
[Contrastive] Epoch 4: Loss = 0.9298
[DEBUG] Epoch 5: feature mean 0.0572, std 0.0252
Converged after 1 iterations.
[Contrastive] Epoch 5: Loss = 1.3947
[DEBUG] Epoch 6: feature mean 0.0574, std 0.0248
Converged after 1 iterations.
[Contrastive] Epoch 6: Loss = 0.6199
[DEBUG] Epoch 7: feature mean 0.0567, std 0.0263
Converged after 1 iterations.
[Contrastive] Epoch 7: Loss = 1.2398
[DEBUG] Epoch 8: feature mean 0.0582, std 0.0229
Converged after 1 iterations.
[Contrastive] Epoch 8: Loss = 1.0848
[DEBUG] Epoch 9: feature mean 0.0585, std 0.0219
Converged after 1 iterations.
[Contrastive] Epoch 9: Loss = 1.5497
[DEBUG] Epoch 10: feature mean 0.0585, std 0.0221
Converged after 1 iterations.
[Contrastive] Epoch 10: Loss = 0.6199

```

Fig. 6: Training log for the 10-class setup. Compact embeddings are observed despite oscillations in contrastive loss.

improvement in accuracy and ARI suggests that pruning long-tail categories reduced noise, but the overall scale of change was small.

E. Evaluation on 10 Classes

In the 10-class configuration, probe accuracy reached **79.90%**, which is very close to the 20-class result. ARI decreased to **0.2188**, while NMI remained almost unchanged at **0.0239**. This shows that reducing classes further did not bring major differences: accuracy improved only marginally and clustering alignment metrics did not benefit.

F. Comparative Analysis

Comparing the three experiments shows that changes in the number of classes produced only minor shifts in evaluation metrics. Accuracy increased slightly as the number of classes decreased, but the overall scale of change from 40 to 10 classes was around 1%. ARI improved when reducing from 40 to 20 classes, but then decreased again at 10 classes, suggesting no consistent trend. NMI remained almost flat across all settings, with values around 0.024.

Overall, altering the dataset size from 40 to 20 to 10 classes did not fundamentally change the behavior of the embeddings. The probe accuracy stayed within a narrow band of less than 2% difference and clustering metrics fluctuated without large improvements.

G. Quantitative Results

The quantitative results are summarized in Table I.

TABLE I: Results across 40, 20, and 10 Class Configurations

Setup	Accuracy (%)	ARI	NMI
40 Classes	78.77	0.2394	0.0242
20 Classes	79.86	0.2633	0.0240
10 Classes	79.90	0.2188	0.0239

V. DISCUSSION

The experiments highlight three main insights. First, dataset balance strongly influences clustering outcomes, as performance metrics improved when long-tail categories were pruned. Second, torque clustering benefits particularly from dense categories where structural similarities are clear, as these provide stronger signals for stable embeddings. Third, enhancements to PointNet embeddings are crucial for stability, enabling torque clustering to converge consistently during training. Despite these strengths, limitations remain, including the reliance on KMeans as the final step and the computational overhead introduced by torque calculations.

VI. CONCLUSION AND FUTURE WORK

We introduced torque clustering as a hybrid unsupervised learning method for point clouds and demonstrated its effectiveness on subsets of the ModelNet40 dataset. Our preprocessing pipeline ensured robustness, while PointNet enhancements stabilized embeddings. Future directions include applying torque clustering to larger-scale datasets (e.g., ShapeNet), integrating graph-based methods, and exploring end-to-end differentiable clustering. Also, the current implementation of TC fixes the number of clusters at 40; future work may also leverage TC's TGap mechanism to enable automatic cluster detection, further improving generalization across datasets without prior knowledge of class counts.

ACKNOWLEDGMENT

The authors would like to thank Md. Sabbir Ahmed and Quazi Ashikur Rahman for guidance, and BRAC University for support.

REFERENCES

- [1] H. Zhu, Y. Wang, D. Huang, W. Ye, W. Ouyang, and T. He, “Point cloud matters: Rethinking the impact of different observation spaces on robot learning,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 77 799–77 830, 2024.
- [2] J. Sun, Q. Zhang, B. Kailkhura, Z. Yu, C. Xiao, and Z. M. Mao, “Benchmarking robustness of 3d point cloud recognition against common corruptions,” *arXiv preprint arXiv:2201.12296*, 2022.
- [3] B. Eckart, W. Yuan, C. Liu, and J. Kautz, “Self-supervised learning on 3d point clouds by learning discrete generative models,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2021, pp. 8248–8257.
- [4] J. Yang and C.-T. Lin, “Autonomous clustering by fast find of mass and distance peaks,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2025.
- [5] M. Afham, I. Dissanayake, D. Dissanayake, A. Dharmasiri, K. Thilakarathna, and R. Rodrigo, “Crosspoint: Self-supervised cross-modal contrastive learning for 3d point cloud understanding,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 9902–9912.
- [6] Y. Ze, N. Hansen, Y. Chen, M. Jain, and X. Wang, “Visual reinforcement learning with self-supervised 3d representations,” *IEEE Robotics and Automation Letters*, vol. 8, no. 5, pp. 2890–2897, 2023.
- [7] Z. Zhang, R. Girdhar, A. Joulin, and I. Misra, “Self-supervised pretraining of 3d features on any point-cloud,” in *Proceedings of the IEEE/CVF international conference on computer vision*, 2021, pp. 10 252–10 263.